

KV Cache的实现和自注意力计算中的QKV

课程简介



报名链接



视频

[📺 KV Cache的实现和自注意力计算中的QKV.mp4](#)

举例

Inputs = "我正在上课"

[32,64,95,77,123]

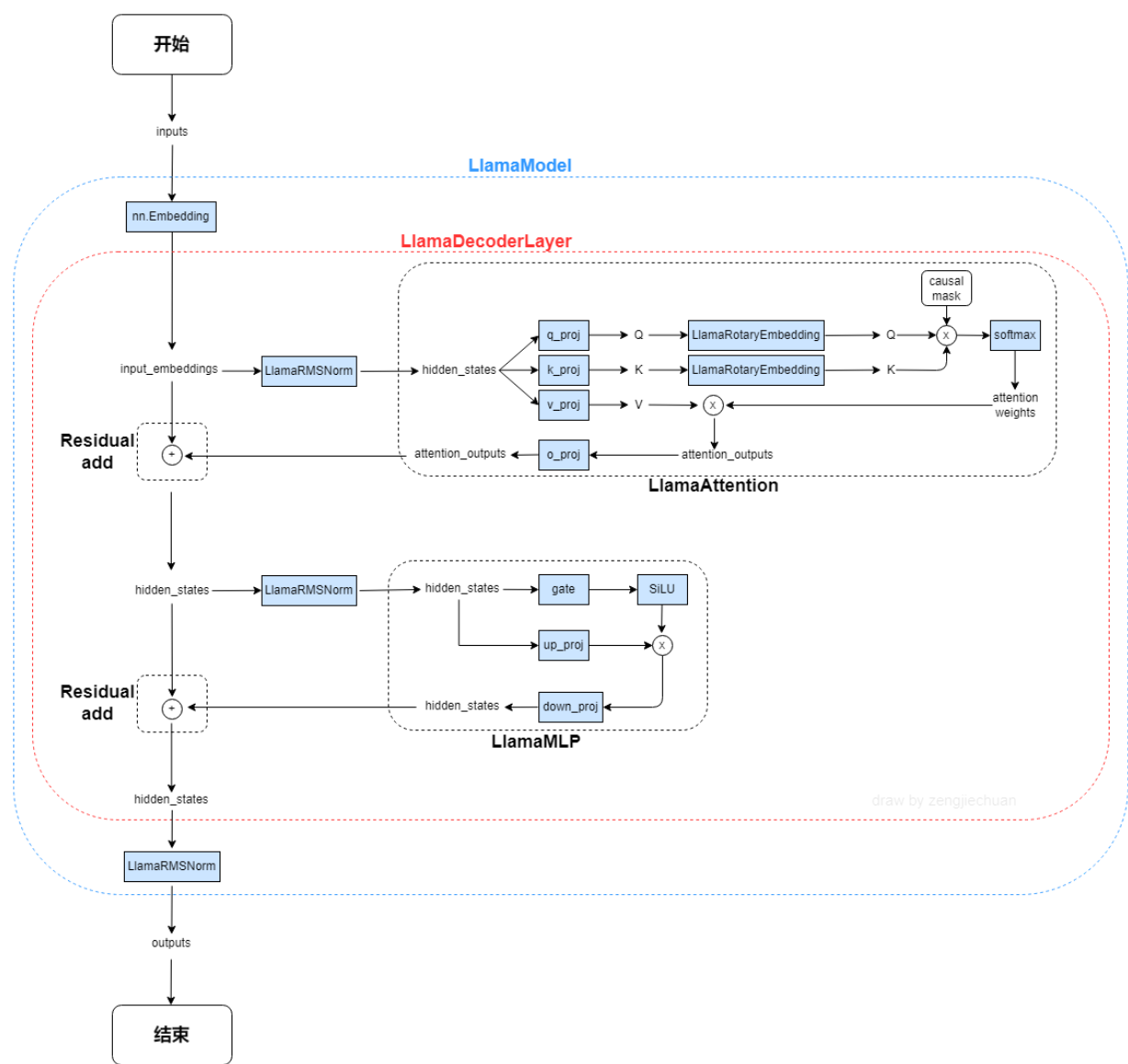
[[512维][512维]]

Inputs = "我"

Token id = 32

通过32在词表中查到了一个512维度的向量

Input embeddings来表示

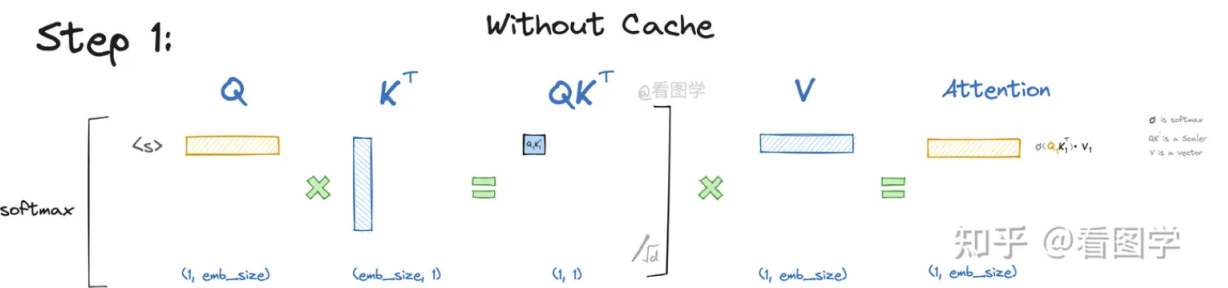


首先，让我们看一下LLama模型的架构图。该模型的输入可以是单段或多段文本。通过Tokenizer工具，例如我们课程项目中使用的sentencepiece，这些文本将被转换成一个词表索引序列。接下来，我们使用嵌入词表，将这些索引序列转换为一组输入向量，我们称之为输入嵌入（input embedding）。然后，我们对这些输入嵌入（input embedding）执行rmsnorm操作。

在此之后，将经过rmsnorm处理的输入嵌入分别与wq、wk和wv这三个矩阵进行矩阵乘法运算，从而得到Q、K、V三个矩阵。我们回归一下上节课中讲到了什么，在上节课中我们讲到了Attention的计算方式以及KV Cache为什么能节省内存，在本节课的一开始，我们先来回顾一下这两点主要内容：

Rmsnorm output @ wq = Q
 Rmsnorm output @ wk = K
 Rmsnorm output @ wv = V
 softmax(Q@K.T)@V

Attention的计算回顾



让我们以第一步的计算为例。在第一步中，特殊符号被编码为数字32。我们使用这个数字作为索引，在词表中查找对应的项。**在词表的第32个位置，记录了一个向量，这个向量代表了开始词的嵌入，其维度为1×dim。**当步长为1时，我们有一个维度为1×dim的input embedding。这里的1表示输入Token的数量，通过将input embedding

与wq和wk矩阵（这两个矩阵的维度均为dim×dim）相乘，分别得到Q和K矩阵。接下来，我们将Q矩阵与K的转置矩阵相乘得到一个注意力权重矩阵。

在步长为1的情况下，Q矩阵的维度为1×dim，K矩阵的维度也是1×dim。当Q矩阵乘以K的转置时，我们得到一个1×1的权重矩阵。然后，这个权重矩阵用于对1×dim的V矩阵进行加权，从而得到最终的注意力输出。同时，input embedding与wv矩阵（维度同样为dim×dim）的乘积结果就是V矩阵。

$$Att_1(Q, K, V) = softmax(Q_1 K_1^T) V_1$$

这里就引申出了三个问题？

1. 怎么实现input embedding乘以wq、wk和wv矩阵；
2. wq、wk和wv矩阵怎么从权重模型文件中提取得到；
3. input embedding和wk 矩阵的乘积k，怎么放入到kv cache中。

目的是求出Rmsnorm output @ wk = K

但是实际上K的前N-1列已经被缓存下来了，我们当前步需要计算的只是第N列。

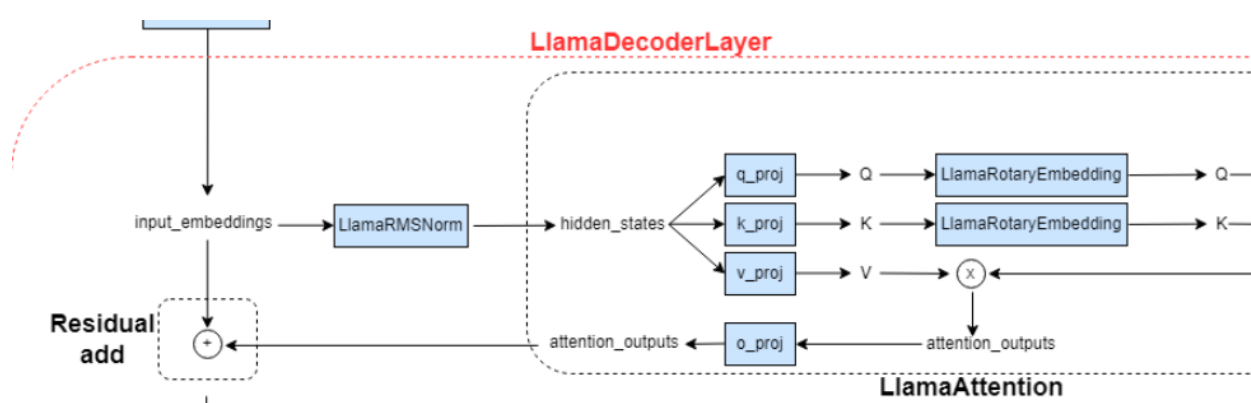
问题1

怎么实现input embedding矩阵乘wq、wk和wv矩阵？

在attention_qkv函数中，我们首先是通过slice_kv_cache函数切分出本步骤中需要切分得到下图中的k和v张量用来存放新一个步骤中的query和value矩阵，上节课已经讲过了这里的步骤，请同学们自行回顾。



另外我们从下图中就可以看出wq@input和wk@input中的输入存放于RMSNorm的输出之中，



因此，在代码的第12行至第15行，我们看到了 `query_layer->forward(rmsnorm_output, query)` 的调用。在这个算子调用过程中，我们将输入的嵌入向量（input embedding）与query layer自身的权重相乘并将计算结果存储在 `query` 变量中。同理，我们也采用了相同的方法来获取键（key）矩阵。最后，在第22

行至第24行，我们对值矩阵进行了相应的计算，就像上文说的那样input embedding来自于RMSNorm层的输出。

$w_q \times \text{input embedding}$

$w_k \times \text{input embedding}$

$w_v \times \text{input embedding}$

```
1 void LLama2Model::attention_qkv(int32_t layer_idx, const
  tensor::Tensor& pos_tensor) const {
2   CHECK(llama_layers_ != nullptr);
3   // kv cache
4   tensor::Tensor query = this-
>get_buffer(ModelBufferType::kQuery);
5   int32_t pos = pos_tensor.index<int32_t>(0);
6   // wq wk wv @ input 此处的key和val就是途中用于存放新一个step中的
  input token @ wk
7   // 和 input token@ kv
8   const auto& [key, val] = slice_kv_cache(layer_idx, pos);
9
10
11  // query
12  // rmsnorm_output @ wq = query
13  const auto& query_layer = llama_layers_-
>wq_layers_.at(layer_idx);
14  CHECK_NE(query_layer, nullptr) << "The query layer in the
  attention block is null pointer.";
15  auto rmsnorm_output =
  get_buffer(ModelBufferType::kOutputRMSNorm);
16  STATUS_CHECK(query_layer->forward(rmsnorm_output, query));
17
18  // key
19  // rmsnorm_output @ wk = key, 就是当前的最后一列k矩阵
20  const auto& key_layer = llama_layers_->wk_layers_.at(layer_idx);
21  CHECK_NE(key_layer, nullptr) << "The key layer in the attention
  block is null pointer.";
22  STATUS_CHECK(key_layer->forward(rmsnorm_output, key));
23
24  // value
25  // rmsnorm_output @ wv = value, 就是当前的最后一行的v矩阵
26  const auto& value_layer = llama_layers_-
>wv_layers_.at(layer_idx);
27  CHECK_NE(value_layer, nullptr) << "The value layer in the
  attention block is null pointer.";
28  STATUS_CHECK(value_layer->forward(rmsnorm_output, val));
29
30  // rope
31  CHECK_NE(llama_layers_->rope_layer_, nullptr)
  << "The RoPE layer in the attention block is null pointer.";
32  STATUS_CHECK(llama_layers_->rope_layer_->forward(
  query, key, pos_tensor,
  get_buffer(ModelBufferType::kSinCache),
  get_buffer(ModelBufferType::kCosCache), tensor::Tensor{}));
33
34 }
35
36 }
```

问题2

wq、wk和wv矩阵的参数怎么从权重模型文件中提取得到？

我们知道模型权重文件是由于 `tools/export.py` 文件导出的，其中`out_file`是我们打开的模型权重参数文件路径，我们依次写入`token_embedding`以及每个`transformer`块中`attention_norm`，`wq`，`wk`和`wv`等参数，所以我们在C++端使用`MMap`打开权重文件后也要用相同的顺序从中读取参数：

```
1  # next write out the embedding weights
2  serialize_fp32(out_file, model.tok_embeddings.weight)
3
4  # now all the layers
5  # attention weights
6  for layer in model.layers:
7      serialize_fp32(out_file, layer.attention_norm.weight)
8  for layer in model.layers:
9      serialize_fp32(out_file, layer.attention.wq.weight)
10 for layer in model.layers:
11     serialize_fp32(out_file, layer.attention.wk.weight)
12 for layer in model.layers:
13     serialize_fp32(out_file, layer.attention.wv.weight)
14 ...
```

其中N是Transformer块的个数，当输出权重模型文件后权重有如下的排布：

```
1  -----
2  token embedding    1 × dim × vocab size
3  -----
4  attention norm     N × dim
5  -----
6  weight query       N × dim × dim <==== pos
7  -----
8  weight key         N × dim × dim
9  -----
10 weight value       N × dim × dim
11 -----
```

```
1  void LLama2Model::create_param_layers() {
2      // 读取embedding layer层的权重
3      // ...
4      // ...
5
6      int32_t dim = config->dim_;
7      // 逐层读取query layer, 开始的偏移是dim × vocab size + N × dim
8      size_t pos = dim * std::abs(config->vocab_size_) + dim *
config->layer_num_;
9      // create weight matrix for query
10     for (int32_t i = 0; i < config->layer_num_; ++i) {
11         auto wq = std::make_shared<op::MatmulLayer>(device_type_,
dim, dim); // 创建一个新的matmul层
12         wq->set_weight(0, {dim, dim}, this->raw_model_data_-
>weight(pos), cpu_device_type);
13         llama_layers->wq_layers_.push_back(wq); // 每个wq layer的
维度是dim×dim
14         pos += dim * dim; // pos指向下一个wq layer权重的开始
15     }
```

本周

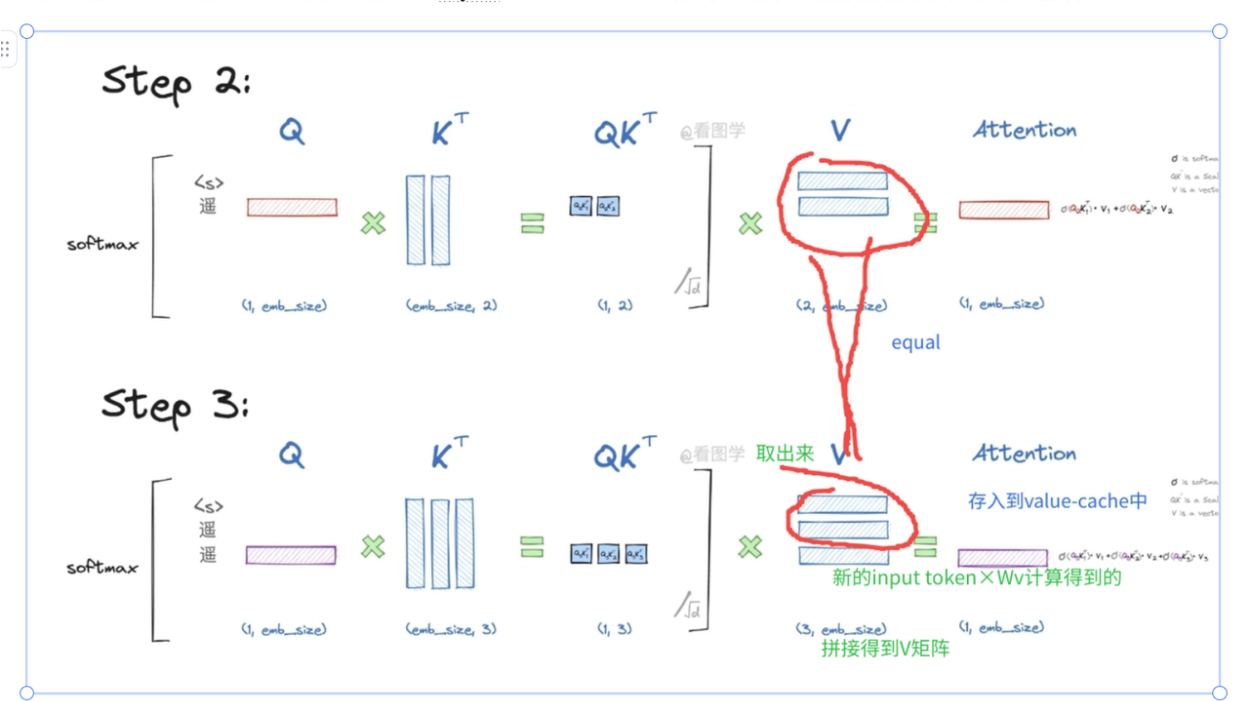
 冯小龙 3月8日 16:25

本步

当在推理开始之前实例化一个wq算子后，接下来就需要从模型权重文件中读取对应的权重，并将其赋值给这个算子，最后在权重赋值完毕后将该算子放到wq_layers_数组中以供推理过程中使用。

问题3

input emebdding和wk矩阵的矩阵乘积k，怎么放入到kv cache中，在上一节课程中我们知道在每次步骤的迭代中，也就是step逐次加1，k矩阵的列数也会加1，而前面的N-1列都是保存在Cache中，只有k矩阵的最后一列是通过**新增加的输入input token**和wk矩阵进行矩阵相乘得到的，我们要做的只需要进行拼接就可以。



所以到调用slice_kv_cache的时候，我们会切出当前transformer块第pos个列作为存放结构的列，对于step3来说，slice_kv_cache切出的列就是第三列，在切出之后就是要将本周新产生的input token，也就是第3个input token与wk矩阵相乘的结果会放到slice_kv_cache新切出的列中。

